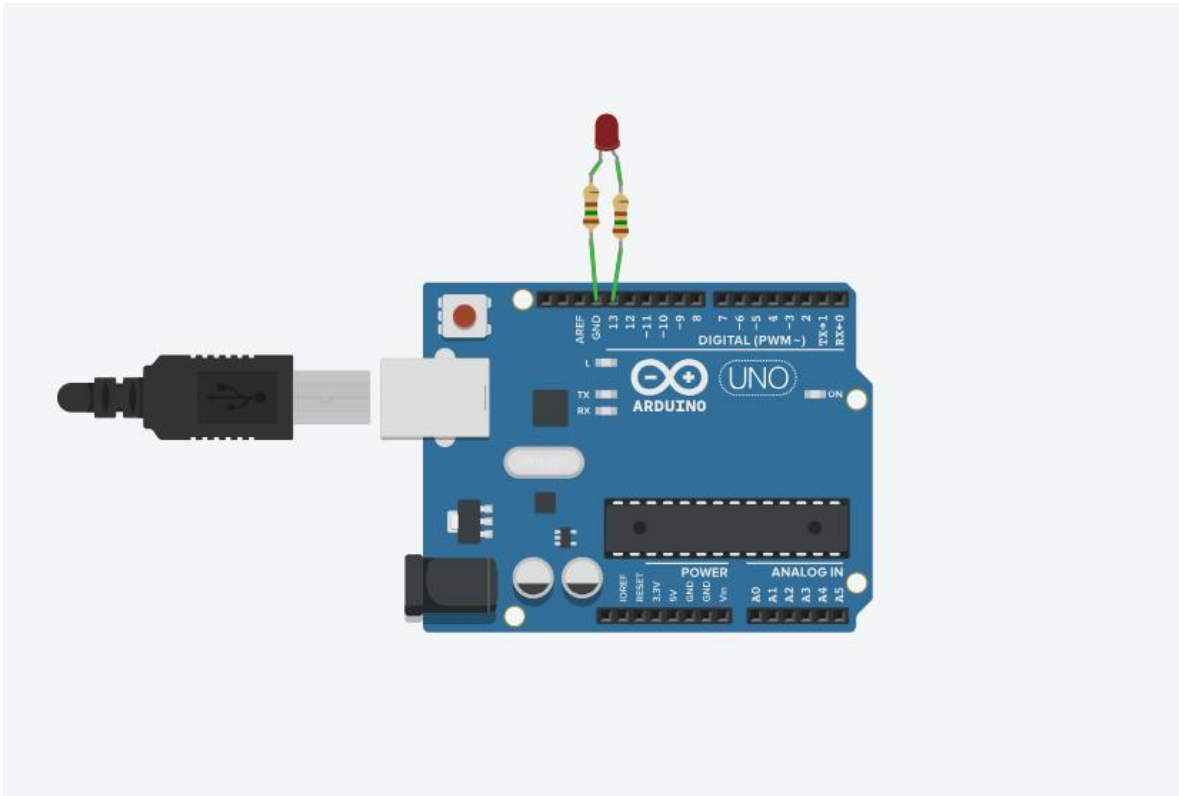


Intermitencia

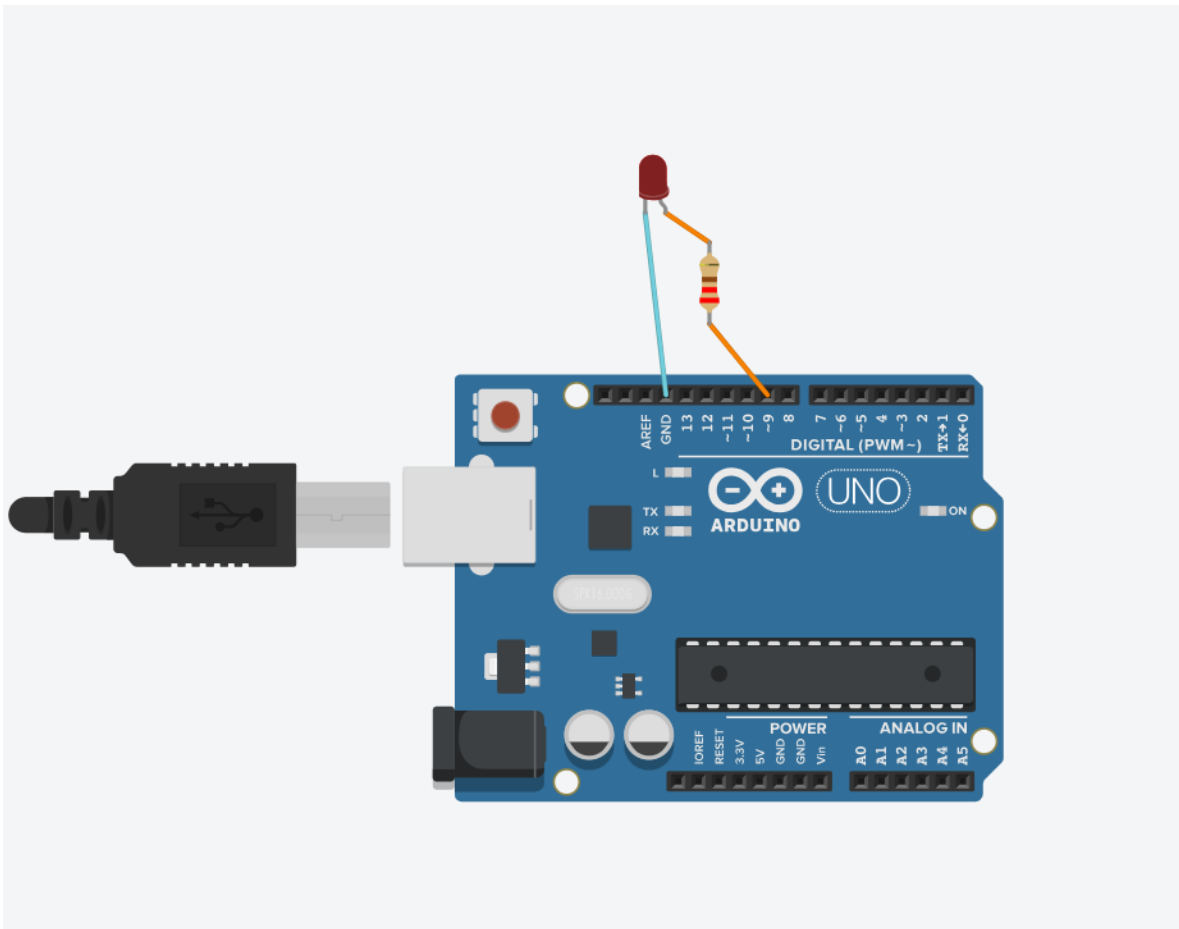


CODIGO:

```
// C++ code
//
void setup()
{
  pinMode(LED_BUILTIN, OUTPUT);
}

void loop()
{
  digitalWrite(LED_BUILTIN, HIGH);
  delay(1000); // Wait for 1000 millisecond(s)
  digitalWrite(LED_BUILTIN, LOW);
  delay(1000); // Wait for 1000 millisecond(s)
}
```

ATENUACION:



```
int ledPin = 9;

int brightness = 0;

int fadeAmount = 5;

void setup() {

  pinMode(ledPin, OUTPUT);

}

void loop() {

  analogWrite(ledPin, brightness);

  brightness = brightness + fadeAmount;

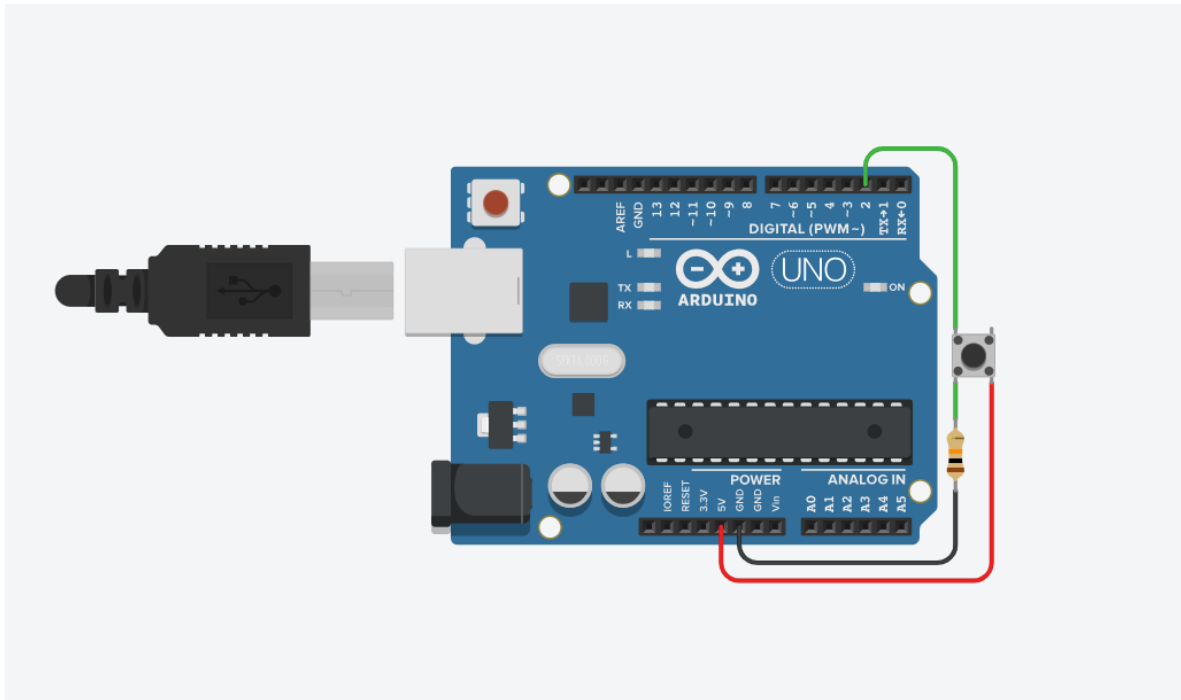
  if (brightness <= 0 || brightness >= 255) {
```

```

    fadeAmount = -fadeAmount;
  }
  delay(30);
}

```

BOTON:



```

int buttonState = 0;

void setup()
{
  pinMode(2, INPUT);
  pinMode(LED_BUILTIN, OUTPUT);
}

void loop()
{
  // read the state of the pushbutton value

```

```

buttonState = digitalRead(2);

// check if pushbutton is pressed. if it is, the

// buttonState is HIGH

if (buttonState == HIGH) {

    // turn LED on

    digitalWrite(LED_BUILTIN, HIGH);

} else {

    // turn LED off

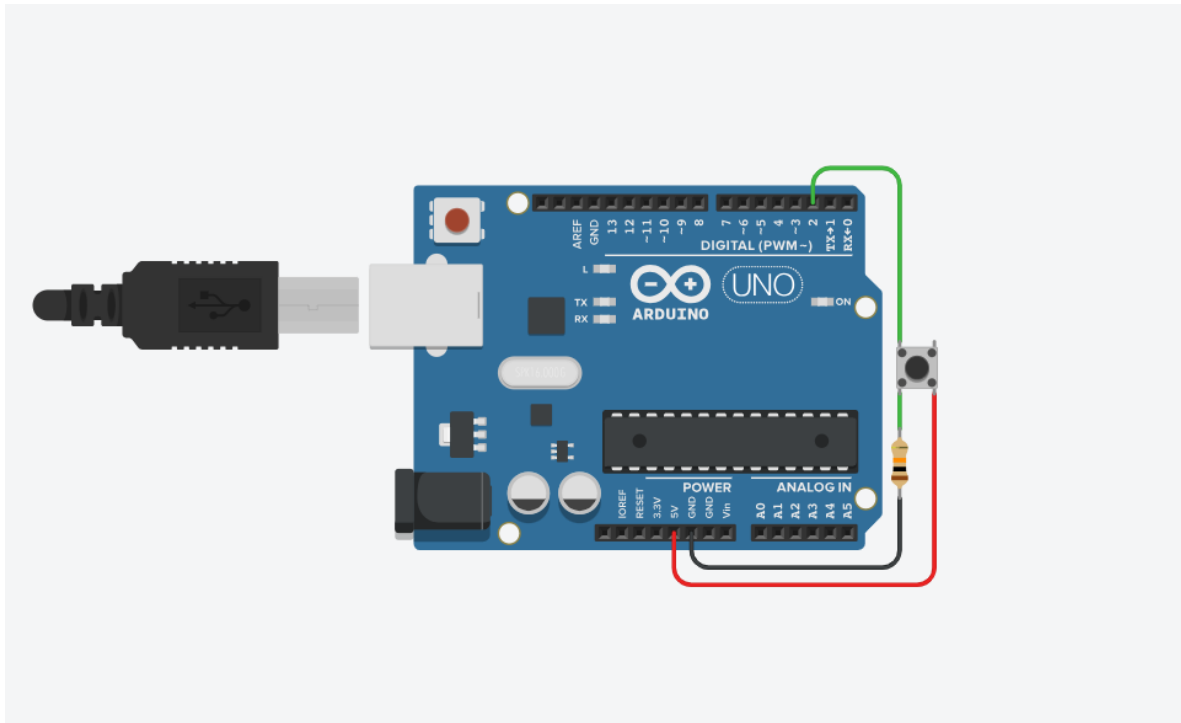
    digitalWrite(LED_BUILTIN, LOW);

}

delay(10); // Delay a little bit to improve simulation performance
}

```

Serie de lectura digital



```
int buttonState = 0;
```

```

void setup()
{
  pinMode(2, INPUT);
  Serial.begin(9600);
}

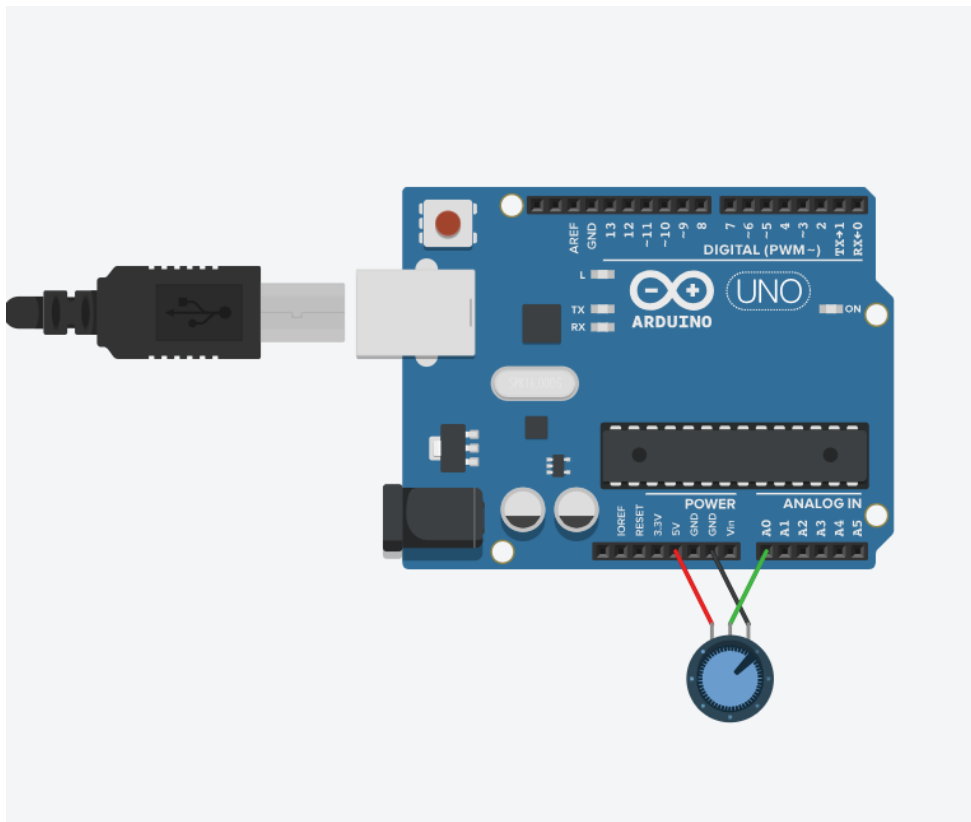
void loop()
{
  // read the input pin
  buttonState = digitalRead(2);

  // print out the state of the button
  Serial.println(buttonState);

  delay(10); // Delay a little bit to improve simulation performance
}

```

ENTRADA ANALOGICA:

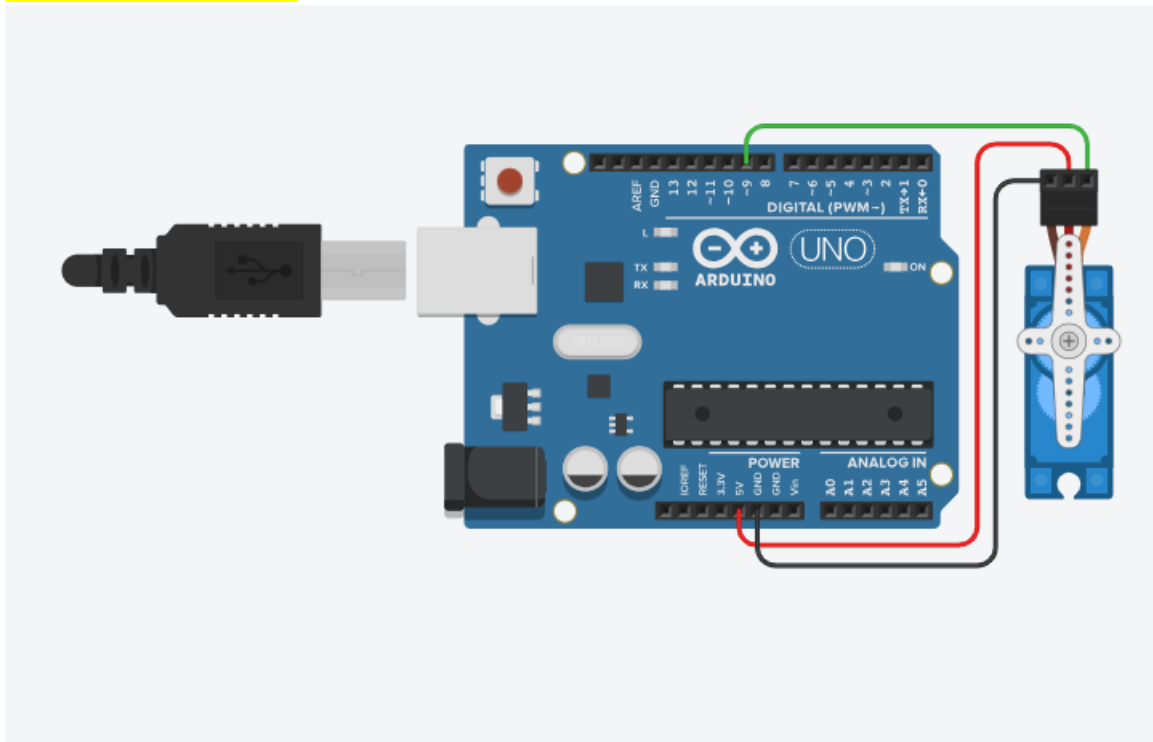


```
int sensorValue = 0;

void setup()
{
  pinMode(A0, INPUT);
  pinMode(LED_BUILTIN, OUTPUT);
}

void loop()
{
  // read the value from the sensor
  sensorValue = analogRead(A0);
  // turn the LED on
  digitalWrite(LED_BUILTIN, HIGH);
  // stop the program for the <sensorValue>
  // milliseconds
  delay(sensorValue); // Wait for sensorValue millisecond(s)
  // turn the LED off
  digitalWrite(LED_BUILTIN, LOW);
  // stop the program for the <sensorValue>
  // milliseconds
  delay(sensorValue); // Wait for sensorValue millisecond(s)
}
```

SERVO MOTOR:



```
#include <Servo.h>
```

```
int pos = 0;
```

```
Servo servo_9;
```

```
void setup()
```

```
{
```

```
  servo_9.attach(9, 500, 2500);
```

```
}
```

```
void loop()
```

```
{
```

```
  // sweep the servo from 0 to 180 degrees in steps
```

```
  // of 1 degrees
```

```
  for (pos = 0; pos <= 180; pos += 1) {
```

```
// tell servo to go to position in variable 'pos'

servo_9.write(pos);

// wait 15 ms for servo to reach the position

delay(15); // Wait for 15 millisecond(s)

}

for (pos = 180; pos >= 0; pos -= 1) {

  // tell servo to go to position in variable 'pos'

  servo_9.write(pos);

  // wait 15 ms for servo to reach the position

  delay(15); // Wait for 15 millisecond(s)

}

}
```